

An Efficient Method for Mining Closed Frequent High Average Utility Occupancy Patterns

Abstract.

Keywords. Frequent high average utility occupancy itemset, Closed itemset, Weak upper-bound, Weak lower bound.

1. Introduction

2. Related work

3. Preliminaries and problem statement

This section introduces the fundamental concepts and notations relevant to mining frequent and high average utility occupancy itemsets. Let $\mathcal{A} \stackrel{\text{def}}{=} \{a_j, j \in J \stackrel{\text{def}}{=} \{1, 2, \dots, M\}\}$ be a set consisting of a finite number of distinct items and A be a subset (*itemset*) of $\mathcal{A}$, $A \subseteq \mathcal{A}$. If A contains exactly k items, then it is referred to as a k -*itemset*, i.e. $k = |A|$. Moreover, assume that, without loss of generality, all items in each itemset are sorted according to a predefined total order relation $<$, e.g. the lexicographic order over items. A profit vector $\mathcal{P} \stackrel{\text{def}}{=} (p(a_j), j \in J \text{ and } p(a_j) > 0)$ indicates the *external* utility of all items in $\mathcal{A}$, where $p(a_j)$ denotes the relative importance of item a_j , such as its unit profit, price, or weight. A pair (a, q) is called a q -item, where $a \in \mathcal{A}$ and q is a positive integer representing the purchase quantity or *internal* utility of item a .

A quantitative database (QDB) $\mathcal{D}$ is a collection of transactions, $\mathcal{D} \stackrel{\text{def}}{=} \{t_i, i \in I \stackrel{\text{def}}{=} \{1, 2, \dots, N\}\}$, where each transaction t_i , defined as $t_i \stackrel{\text{def}}{=} \{(a_j, q_{ij}), j \in J_i\}$ including a set of q -items, is uniquely identified by a transaction identifier ($TID = i$), and J_i is an index subset of the J set, $J_i \subseteq J$. The *utility* of a q -item (a, q) is denoted and defined as $u(a, q) \stackrel{\text{def}}{=} q * p(a)$. In simpler terms, it represents the profit earned from selling item a when the transaction contains the q -item (a, q) . To avoid redundant calculations of the utility u for q -items (a_j, q_{ij}) across transactions t_i of $\mathcal{D}$, these values are computed only once. The result is stored in a new corresponding QDB, called the *integrated* QDB according to $\mathcal{D}$ and $\mathcal{P}$, denoted and defined as $\mathcal{D}' \stackrel{\text{def}}{=} \{t_i \stackrel{\text{def}}{=} \{(a_j, q'_{ij}), j \in J_i, i \in I\}$, where $q'_{ij} = q_{ij} * p(a_j)$ if $(a_j, q_{ij}) \in t_i, \forall i \in I, j \in J_i$.

Definition 1 (*Operator ρ and support of an itemset* [40]).

- For any non-empty itemset $A \subseteq \mathcal{A}$, let $\rho(A) \stackrel{\text{def}}{=} \{t \in \mathcal{D}' \mid t \supseteq A\}$ be the set of transactions in $\mathcal{D}'$ that contain A , where $t \supseteq A$ means that each item a_j of A must appear in a q -item (a_j, q_{ij}) of t (or t is called a supporting transaction of A). By convention, $\rho(\emptyset) \stackrel{\text{def}}{=} \mathcal{D}'$.
- The support of an itemset A , denoted as $\text{supp}(A)$, is given by the number of transactions in $\mathcal{D}'$ containing A , i.e. $\text{supp}(A) = |\rho(A)|$.
- The set of all non-empty itemset A that have at least one occurrence in the transactions of $\mathcal{D}'$ is denoted and defined as $\mathcal{JS} \stackrel{\text{def}}{=} \{A \subseteq \mathcal{A} \mid (A \neq \emptyset) \wedge (\rho(A) \neq \emptyset)\}$.

Running example. For example, let us consider a QDB $\mathcal{D}$ consisting of 6 transactions $t_1 = \{(a, 2), (b, 5), (c, 56), (d, 25), (e, 14), (f, 4)\}$, $t_2 = \{(b, 2), (c, 2), (e, 80), (f, 8)\}$, $t_3 = \{(b, 60), (e, 30), (f, 2)\}$, $t_4 = \{(a, 5), (d, 5)\}$, and $t_5 = \{(a, 2), (b, 6), (c, 3), (d, 2), (e, 55), (f, 10)\}$, and the profit vector $\mathcal{P} = (2; 4; 2; 4; 3; 2)$ of 6 items a, b, c, d, e and f . The integrated QDB $\mathcal{D}'$ according to $\mathcal{D}$ and $\mathcal{P}$, shown in Table 1, will be used as the running example. It is observed that there are 2 transactions containing bc , $\rho(bc) = \{t_1, t_2, t_5\}$, so $\text{supp}(bc) = 2$.

Definition 2 (*Transaction utility, utility [4] and average utility [3] of an itemset in a transaction*). Let us consider any transaction $t \in \mathcal{D}'$.

- The utility of item a in t (i.e. $\exists (a, q) \in t$) and the transaction utility of t are respectively defined as $u(a, t) \stackrel{\text{def}}{=} q$ and $u(t) \stackrel{\text{def}}{=} \sum_{(a, q) \in t} u(a, t)$.
- For any itemset A in t ($A \subseteq t$, i.e. all items of A appear in t), the *utility* and *average utility* of A in t are denoted

and defined as $u(A, t) \stackrel{\text{def}}{=} \sum_{a \in A} u(a, t)$ and $au(A, t) \stackrel{\text{def}}{=} \frac{u(A, t)}{|A|}$, respectively, where $|A|$ denotes the length (or the number of items) of A .

Table 1. The integrated QDB $\mathcal{D}'$.

<i>TID</i>	<i>Transaction (item, utility)</i>
t_1	$(a, 4), (b, 20), (c, 112), (d, 100), (e, 42), (f, 8)$
t_2	$(b, 8), (c, 4), (e, 240), (f, 16)$
t_3	$(b, 240), (e, 90), (f, 4)$
t_4	$(a, 10), (d, 20)$
t_5	$(a, 4), (b, 24), (c, 6), (d, 8), (e, 165), (f, 20)$

Definition 3 (*Utility occupancy* [13], *average utility occupancy* [17] *of itemset*).

- The *utility occupancy* of an itemset A in a supporting transaction $t \in \mathcal{D}'$, $A \subseteq t$, is defined as the *relative utility* of A in t compared to the transaction utility of t , given by $uo(A, t) \stackrel{\text{def}}{=} \frac{u(A, t)}{u(t)}$.
- The average of the *utility occupancies* of A across all its supporting transactions $t \in \rho(A)$ is called the *utility occupancy* of A in the entire $\mathcal{D}'$, denoted and defined as $uo(A) \stackrel{\text{def}}{=} \frac{1}{\text{supp}(A)} \sum_{t \in \rho(A)} uo(A, t)$.
- The *average utility occupancy* of A (in $\mathcal{D}'$) is denoted and defined as $auo(A) \stackrel{\text{def}}{=} \frac{uo(A)}{|A|}$. If we define $auo(A, t) \stackrel{\text{def}}{=} \frac{uo(A, t)}{|A|} = \frac{au(A, t)}{u(t)}$, then $auo(A)$ can be represented in another form as $auo(A) = \frac{1}{\text{supp}(A)} \sum_{t \in \rho(A)} auo(A, t)$.

$$\text{Indeed, } auo(A) = \frac{uo(A)}{|A|} = \frac{\frac{1}{\text{supp}(A)} \sum_{t \in \rho(A)} uo(A, t)}{|A|} = \frac{1}{\text{supp}(A)} \sum_{t \in \rho(A)} \frac{uo(A, t)}{|A|} = \frac{1}{\text{supp}(A)} \sum_{t \in \rho(A)} auo(A, t).$$

Obviously, for any $A \in \mathcal{IS}$, since $auo(A, t) \in (0; 1], \forall t \in \rho(A)$, we always have $auo(A) \in (0; 1]$.

Example 1. For example, consider itemset $A = bc$, the utility and average utility of A in t_1 are respectively $u(A, t_1) = 20 + 112 = 132$ and $au(A, t_1) = \frac{132}{2} = 66$. Since the transaction utility of t_1 is $u(t_1) = 286$, the utility occupancy of A in t_1 is $uo(A, t_1) = \frac{u(A, t_1)}{u(t_1)} = \frac{132}{286} = 0.46$. Thus, the average utility occupancy of A (with length 2) in t_1 , $auo(A, t_1) = \frac{0.46}{2} = 0.23$. Similarly, by considering all transactions containing A , $\rho(A) = \{t_1, t_2, t_5\}$, the average utility occupancy of A is determined as follows: $auo(A) = (0.23 + (\frac{12}{268} + \frac{30}{227})/2)/3 = 0.106$.

Definition 4 (*Frequent high average-utility occupancy itemset*). Given two user-specified positive *minimum average utility occupancy* and *support* thresholds, denoted respectively as mo and ms , an itemset A is called a *high average-utility occupancy itemset* (HAUOI) if its average-utility occupancy in $\mathcal{D}$ is not less than mo , $auo(A) \geq mo$. Otherwise, $auo(A) < mo$, A is called a *low average-utility occupancy itemset* (LAUOI). An HAUOI A is said to be a *frequent HAUOI* (FHAUOI) if its support is not less than ms , $\text{supp}(A) \geq ms$. The problem of FHAUOI mining ($\mathcal{FHAUOIM}$) is to discover the set $\mathcal{FHAUOI} \stackrel{\text{def}}{=} \{A \in \mathcal{IS} \mid auo(A) \geq mo \wedge \text{supp}(A) \geq ms\}$ of all FHAUOIs.

Example 2. Consider itemset $A = bc$ in Example 1, $ms = 1$ and $mo = 0.1$. Since $auo(A) = 0.106 > mo$ and $\text{supp}(A) = 3 > ms$, the A itemset is an FHAUOI.

Definition 5 (*Set CFHAUOI*). Let A be an FHAUOI, $A \in \mathcal{FHAUOI}$. A is called a *closed frequent high average-utility occupancy itemset* (CFHAUOI) if there exists no FHAUOI being a proper superset of A and has the same support. Then, let $\mathcal{CFHAUOI}$ denote the set of all CFHAUOIs, defined as

$$\mathcal{CFHAUOI} \stackrel{\text{def}}{=} \{A \in \mathcal{FHAUOI} \mid \nexists B \in \mathcal{FHAUOI}: B \supset A \wedge \text{supp}(B) = \text{supp}(A)\}.$$

Problem statement. Given a QDB $\mathcal{D}'$ and user-defined positive thresholds for minimum average utility occupancy mo and minimum support ms , the task of closed frequent high average utility occupancy itemset

mining ($\mathcal{CFH}\mathcal{AUOIM}$) involve identifying the sets $\mathcal{CFH}\mathcal{AUOI}$.

Example 3.

Definition 6 (*Forward extension of an itemset* [32]). Assume that all items in $\mathcal{A}$ are sorted according to the total order relation $<$. For any item y and two itemsets A and B , let us denote $A < y$ iff $x < y, \forall x \in A$, or $A < B$ iff $x < y, \forall x \in A, \forall y \in B$.

- b. When $A \cap B = \emptyset$, $A \cup B$ is denoted as $A + B$, which is the union of two disjoint itemsets. If $A \cap B = \emptyset$ and $A < B$, then the notation $A + B$ is replaced by the notation $A \oplus B$, and the itemset $S = A \oplus B$ is said to be a *forward extension* (FE), or simply an extension, of A with B . Especially, the FE $A \oplus y$ of A with item y (where $y > A$) is succinctly denoted as Ay .
- c. If $B \neq \emptyset$, then $S = A \oplus B$ is called a *proper FE* (pFE) of A . In other words, an FE of the itemset A is either A (when $B = \emptyset$) or a pFE of A .

4. Mining closed frequent high average utility occupancy itemsets

4.1. Pruning of low average utility occupancy itemsets

To efficiently reduce the search space, it is essential to leverage the *anti-monotonicity* ($\mathcal{AM}$) inherent in the support function, along with upper bounds on auo . For instance, if an itemset A is infrequent then all its super-itemsets are also infrequent, and as a result, the entire branch $Br(A)$, which contains only infrequent extensions of A , can be pruned.

Definition 7 (*Upper bound and anti-monotonicity* [31]). Let $\mathbb{R}_+$ denote the set of all positive real numbers.

- a. A function $\hat{u}: \mathcal{IS} \rightarrow \mathbb{R}_+$ of itemsets is called an *upper bound* (UB) on auo , denoted as $auo \ll \hat{u}$, if $auo(A) \leq \hat{u}(A), \forall A \in \mathcal{IS}$.
- b. For any two functions f_1 and f_2 of itemsets, f_1 is called *tighter* than f_2 , denoted as $f_1 \ll f_2$, if $f_1(A) \leq f_2(A), \forall A \in \mathcal{IS}$.
- c. A function $g: \mathcal{IS} \rightarrow \mathbb{R}_+$ is called *anti-monotonic* ($\mathcal{AM}$), denoted as $\mathcal{AM}(g)$, if $g(B) \leq g(A), \forall B \supseteq A$.

It is well-known that $supp$ is $\mathcal{AM}$. However, since the $\mathcal{AM}$ property does not hold for auo , it is necessary to design $\mathcal{AM}$ UBs on auo . Inspired by the $\mathcal{AM}$ UB $auoub$ on auo [17], which represents the maximum item utility occupancy, we generalize it and develop a tighter new $\mathcal{AM}$ UB, named $lauo_\rho$. Its value is the average of the top- k largest item utility occupancies. The $\mathcal{AM}$ property of $supp$ and $lauo_\rho$ can be utilized to remove *infrequent* or *LAUO branches* from the prefix search tree.

Definition 8. Let $\mathcal{O} \stackrel{\text{def}}{=} (o_1, \dots, o_n)$ be a vector of n objects, and each object o_i has a corresponding value $v_i \stackrel{\text{def}}{=} v(o_i)$, for any $i: 1 \leq i \leq n$. After sorting the objects of $\mathcal{O}$ in *descending* order of their values, we obtain the new descending vector: $(o_{i_1}, \dots, o_{i_n})$, where $v_{i_1} \geq v_{i_2} \geq \dots \geq v_{i_n}$, and $1 \leq i_h \leq n$ for any $h = 1, \dots, n$. The subset of k objects with the k largest v -values is defined as $\mathcal{T}_k(\mathcal{O}, v) \stackrel{\text{def}}{=} (o_{i_1}, \dots, o_{i_k})$, where $1 \leq k \leq n$ and $v \stackrel{\text{def}}{=} \{v_i, 1 \leq i \leq n\}$. In particular, $\mathcal{T}_k(t, u) = (a_{i_1}, \dots, a_{i_k})$ consists of the k -largest q -items (a_{i_j}, u_{i_j}) of transaction $t \in \mathcal{D}'$ with descending utilities: $u_{i_1} \geq u_{i_2} \geq \dots \geq u_{i_k}$.

Definition 9 (*Upper bound $lauo_\rho$ on auo*). For any frequent itemset $A \in \mathcal{FI}$,

- a. For any itemset $A \in \mathcal{FI}$ and transaction $t \in \rho(A)$, the k -largest average utility of t is defined as $lau(t, k) \stackrel{\text{def}}{=} \frac{1}{k} \sum_{(a,q) \in \mathcal{T}_k(t,u)} q$ for $1 \leq k \leq |t|$, and $lauo(A, t) \stackrel{\text{def}}{=} \frac{lau(t, |A|)}{u(t)}$.
- b. An upper bound (UB) on auo , called $lauo_\rho$ (*largest average utility occupancy based on ρ*), is defined as $lauo_\rho(A) \stackrel{\text{def}}{=} \frac{1}{ms} \sum_{t \in \mathcal{T}_{ms}(\rho(A), lauo)} lauo(A, t)$.

Example 4. For $ms = 2$, consider itemset $A = bc$ with $supp(A) = 3$, $length(A) = 2$ and $\rho(A) = \{t_1, t_2, t_5\}$. The computation of the new UB $lauo_\rho(A)$ is as follows. First, for transaction t_1 , we have $lauo(A, t_1) = (112 + 100)/2/286 = 0.37$ because 112 and 100 are the two largest utilities of t_1 . Similarly, for the two remaining

¹ iff means “if and only if”.

transactions t_2 and t_5 , $lauo(A, t_2) = (240 + 16)/2/268 = 0.48$ and $lauo(A, t_5) = (165 + 24)/2/227 = 0.42$. Since $lauo(A, t_2) > lauo(A, t_5) > lauo(A, t_1)$, we have $\mathcal{T}_2(\rho(A), lauo) = \{t_2, t_5\}$, and $lauo_\rho(A) = (0.48 + 0.42)/2 = 0.45$.

Proposition 1 (*The $\mathcal{AM}$ properties of $supp$ and UB $lauo_\rho$*).

- a. $lauo_\rho$ is a UB on auo , i.e. $auo(A) \leq lauo_\rho(A), \forall A \in \mathcal{FI}$.
- b. $lauo_\rho$ is $\mathcal{AM}$, meaning $lauo_\rho(B) \leq lauo_\rho(A)$ for any super-itemset B of A in $\mathcal{FI}$, $B \supseteq A$.

Example 5. For $A = bc$ and $mo = 0.5$, the $Br(A)$ branch can be eliminated utilizing the new UB $lauo_\rho$ since $lauo_\rho(A) = 0.45 < mo$. For example, $auo(A) = 0.106 < lauo_\rho(A) < mo$. Hence, the tighter UB $lauo_\rho$ will be used in the pruning strategies, as stated in Theorem 1 below.

By using the $\mathcal{AM}$ property of $supp$ and the UB $lauo_\rho$ on auo , we design two pruning strategies: (a) *reducing* the QDB $\mathcal{D}'$ and (b) *tightening* the candidate item sets for extensions. Let $PC(A) \stackrel{\text{def}}{=} (supp(A) < ms) \vee (lauo_\rho(A) < mo)$ represent the *pruning condition* used to check whether A is an infrequent or LAUO itemset. Since the functions $supp$ and $lauo_\rho$ are $\mathcal{AM}$, the necessary condition for any FE of A to be an FHAUOI is *not*($PC(A)$). Utilizing the pruning condition PC during the extension of itemset A helps prevent the generation of much larger itemsets and significantly reduces memory and time consumption in the mining process. Let $ExtIts(A)$ denote the set of candidate items for extending itemset A .

Theorem 1 (*Pruning strategy 1 based on $supp$ and $lauo_\rho$ - PSI*). For any item $y \in \mathcal{A}$, if $PC(y)$ is true, then this irrelevant item can be safely removed from the database $\mathcal{D}'$. Furthermore, for any itemset A , if $PC(A)$ holds, the entire branch $Br(A)$ can also be pruned from the search tree.

Definition 10 (*Weak upper bound*). A function $\hat{w}$ of itemsets is called a *weak upper bound* (WUB) on auo , denoted as $auo \propto \hat{w}$, if $auo(B) \leq \hat{w}(A)$ for any *frequent* pFE $B = A \oplus E$ of A with $E \neq \emptyset$.

Let $\delta(A)$ denote the set of all transactions that contain at least one pFE of A , that is $\delta(A) \stackrel{\text{def}}{=} \{t \in \mathcal{D}' | t \supseteq A \oplus E \supset A, E \neq \emptyset\}$. Assuming that there exists a *frequent* pFE $A \oplus E$ of A in a transaction $t \in \mathcal{D}'$ with $E \neq \emptyset$. Therefore, $\delta(A) \supseteq \{t \in \mathcal{D}' | t \supseteq A \oplus E\} = \rho(A \oplus E)$, so $|\delta(A)| \geq supp(A \oplus E) \geq ms$. Otherwise, if no such frequent pFE $A \oplus E$ of A is found, the extension process for A is stopped. This observation leads to the following concept: A is called a *strictly frequent* itemset if $|\delta(A)| \geq ms$. Henceforth, we will focus only on *strictly frequent* itemsets.

A WUB $\hat{w}(A)$ on $auo(A)$ is often provided by relying on WUBs $\hat{w}(A, t)$ on $auo(A, t)$ for each transaction $t \in \delta(A)$, i.e. $auo(B, t) \leq \hat{w}(A, t)$ for any *frequent* pFE $B = A \oplus E$ of A in t . Thus, an interesting question arises: Is there a WUB $\hat{w}(A, t)$ on $auo(A, t)$ in each supporting transaction t that is the tightest (or smallest)? The answer is affirmative. Inspired by the WUB $mrao$ on the au function [9], this paper first proposes a new WUB $tauo(A, t)$ on $auo(A, t)$ for each transaction t in $\delta(A)$. Proposition 2.b below ensures that it is the *tightest* WUB for each transaction. This means that $tauo(A, t) \leq wub(A, t)$ for any WUB $wub(A, t)$ on $auo(A, t)$ in each transaction $t \in \delta(A)$. By averaging $tauo(A, t)$ across all supporting transactions t in $\delta(A)$, we can derive the WUB $tauo(A)$ on $auo(A)$, as shown in Definition 12.

For any itemset A and transaction $t \in \delta(A)$, denote $l \stackrel{\text{def}}{=} |A|$ and $r \stackrel{\text{def}}{=} |t| - l \geq 1$. Let $rem(A, t)$ be the set of all r remaining q -items in transaction t after A . After sorting the q -items of $rem(A, t)$ in *descending* order of utilities, we obtain the series $\mathcal{R}^{A,t} = \mathcal{T}_r(rem(A, t), u)$ (see Definition 8), denoted simply as $\{(a_j, q_j \stackrel{\text{def}}{=} u(a_j, t)), j = 1..r\}$. For brevity, denote $u \stackrel{\text{def}}{=} u(A, t)$, $au \stackrel{\text{def}}{=} au(A, t) = u/l$, $s_0 \stackrel{\text{def}}{=} au$ and $s_k \stackrel{\text{def}}{=} \frac{u + \sum_{j=1..k} q_j}{l+k}$, for $1 \leq k \leq r$. Here, s_k represents the *maximum* average utility of all pFEs $B = A \oplus E$ of A in t with $|E| = k \geq 1$. Then, $\max\{s_k, 1 \leq k \leq r\}$ represents the maximum average utility of all possible pFEs of A in t , and the tightest WUB on $auo(A, t)$ should be this value, as presented in **Error! Not a valid bookmark self-reference.** and confirmed by Proposition 2.

Definition 11 (*The tightest WUB $tauo(A, t)$ for each supporting transaction t*). Assuming that the following hypothesis (H) holds: there exists a *pivot* (with $1 \leq pivot \leq r$) that serves as the pivot index in $\mathcal{R}^{A,t}$, satisfying $((\forall j = 1..pivot: q_j > s_{j-1}) \wedge (pivot = r \vee (pivot < r \wedge q_{pivot+1} \leq s_{pivot})))$. Conversely, if (H) does not

hold, then by convention, we set $pivot \stackrel{\text{def}}{=} 1$. Let $\tau(A, t) \stackrel{\text{def}}{=} \frac{u + \sum_{j=1, \dots, pivot} q_j}{l + pivot}$. Then, the tightest WUB $\tau(A, t)$ on $auo(A, t)$ in each transaction $t \in \delta(A)$ is denoted and defined as $\tau(A, t) \stackrel{\text{def}}{=} \frac{\tau(A, t)}{u(t)}$.

Example 6. For $A = bc$, let's examine $t_1 \in \delta(A) = \{t_1, t_2, t_5\}$. Then, we obtain $rem(A, t_1) = \{(d, 100), (e, 42), (f, 8)\}$, so $r = 3$, $\mathcal{R}^{A, t_1} = \{(d, q_1 = 100), (e, q_2 = 42), (f, q_3 = 8)\}$, and $s_0 = au(A, t_1) = \frac{u(A, t_1)}{2} = 66$, $s_1 = \frac{u(A, t_1) + 100}{2+1} = 77.3$, $s_2 = \frac{u(A, t_1) + 100 + 42}{2+2} = 68.5$ and $s_3 = \frac{u(A, t_1) + 100 + 42 + 8}{2+3} = 56.4$. Since $s_0 < q_1$ and $q_2 = 42 < s_1$, we have $pivot = 1$ and $\tau(A, t_1) = s_1 = 77.3$, so $\tau(A, t_1) = 77.3/286 = 0.27$.

For any itemset A and each fixed transaction $t \in \delta(A)$, a function $\hat{w}(A, t)$ is said to be a WUB on $auo(A, t)$, denoted as $auo(A, t) \bowtie \hat{w}(A, t)$, if $auo(B, t) \leq \hat{w}(A, t)$ for any frequent pFE B of A in t . For any two functions $\hat{w}_1(A, t)$ and $\hat{w}_2(A, t)$, $\hat{w}_1(A, t)$ is called *tighter* than $\hat{w}_2(A, t)$, denoted as $\hat{w}_1(A, t) \ll \hat{w}_2(A, t)$, if $\hat{w}_1(A, t) \leq \hat{w}_2(A, t)$, $\forall t \in \delta(A)$. A WUB $\hat{w}^*(A, t)$ is considered the *tightest* WUB on $auo(A, t)$ in each t if for any WUB $\hat{w}(A, t)$ on $auo(A, t)$, $\hat{w}^*(A, t) \leq \hat{w}(A, t)$, $\forall t \in \delta(A)$.

Proposition 2 (*The tightest WUB $\tau(A, t)$ on $auo(A, t)$*). For any itemset A and transaction $t \in \delta(A)$, then

- $\tau(A, t) = \max\{s_k, 1 \leq k \leq r\}$.
- $\tau(A, t)$ is the *tightest* WUB on $auo(A, t)$ in each transaction t .

Based on the tightest WUB $\tau(A, t)$ for each transaction t , the WUB $\tau_\delta(A)$ on $auo(A)$ is designed as follows.

Definition 12 (*Weak upper bound τ_δ on auo*). For any *strictly* frequent itemset $A \in \mathcal{FI}$, $|\delta(A)| \geq ms$, a new WUB on auo is denoted and defined as $\tau_\delta(A) \stackrel{\text{def}}{=} \frac{1}{ms} \sum_{t \in \mathcal{F}_{ms}(\delta(A), \tau(A))} \tau(A, t)$;

Example 7. For itemset $A = bc$ and $ms = 2$ in Example 6, we have $\rho(A) = \{t_1, t_2, t_5\}$. Then, $\tau(A, t_1) = 0.27$, $\tau(A, t_2) = 0.31$ and $\tau(A, t_5) = 0.29$. Hence, $\tau_\delta(A) = \frac{0.31 + 0.29}{2} = 0.3 < lauo_\rho(A) = 0.45$.

The following Theorem 2 states that τ_δ is the *tightest* WUB among all previously proposed (W)UBs on auo .

Theorem 2 (*Properties of the WUB τ_δ*).

- τ_δ is a WUB on auo .
- τ_δ is tighter than $lauo_\rho$.

Thus, τ_δ is the tightest WUB among all previously proposed (W)UBs on auo .

The properties of τ_δ lead to a highly efficient strict depth pruning strategy, as stated in Theorem 3.

Theorem 3 (*Pruning strategy 2 based on τ_δ - PS2*). For any frequent itemset $A \in \mathcal{FI}$, if $\tau_\delta(A) < mo$, the *proper* branch $pBr(A)$ can be pruned earlier, even if A is an HAUO itemset. Additionally, if $au(A) < mo$, the entire branch $Br(A)$ can be safely pruned.

Example 8. For $mo = 0.4$, consider itemset $A = bc$ as in Example 7. By applying the $\mathcal{AM}$ UB $lauo_\rho$, the branch $Br(A)$ fails to be pruned. However, the strategy $\mathcal{SDPS}(\tau_\delta)$, which leverages the proposed tightest WUB τ_δ from Theorem 3, successfully prunes the *proper* branch $pBr(A)$.

4.2. Pruning of non-closed FHAUOIs

For brevity, for any itemset B , if $B \notin \mathcal{FH\mathcal{AUOI}}$ (the set of all FHAUOIs) or B is not a CFHAUOI (Closed frequent high average-utility occupancy itemsets), then B is referred to as a *non-closed FHAOU (NCFHAOU)* itemset. Similarly, if the $Br(B)$ branch, which includes B and all its FEs, has no CFHAUOIs, it is referred to as an *NCFHAOU branch*.

It is worth noting that the WUBs presented above are designed specifically to swiftly eliminate LAUOIs. However, even with their application, the discovery of FHAUOIs can still result in a substantial number, particularly for extensive QDBs or low values of mo and ms used. Consequently, discovering CFHAUOIs from the entire set of FHAUOIs based solely on their definition presents a challenge, because algorithms must verify

the inclusion relationship among FHAUOIs, which can be complicated. To address this issue, we need to design *weak lower bounds* (WLB) or *lower bounds* (LB) on *au*, which will facilitate the development of efficient strategies for the early elimination of *NCFHAUO* itemsets or branches.

Definition 1 (WLB and LB on *auo*). For any transaction $t \in \mathcal{D}$ and itemset A , a function lb is said to be an LB on *auo* if $lb(A) \leq auo(A)$. A function $wlb(A, t)$ is called a WLB on *auo*(A, t) if $wlb(A, t) \leq auo(C, t)$ for any proper FE C of A . Similarly, wlb is referred to as a WLB on *au* if $wlb(A) \leq auo(C)$ for any proper FE C of A .

For any itemset A and transaction $t \in \delta(A)$, denote $l \stackrel{\text{def}}{=} |A|$. Let $rem(A, t)$ be the set of all r remaining q -items in transaction t after A . After sorting the q -items of $rem(A, t)$ in *ascending* order of utilities, we obtain the series $X \stackrel{\text{def}}{=} \{x_1, \dots, x_r\}$ with $r = |X|$. For brevity, denote $u \stackrel{\text{def}}{=} u(A, t)$, $au_0 \stackrel{\text{def}}{=} au(A, t) = u/l$, and $au_k \stackrel{\text{def}}{=} \frac{u + \sum_{j=1..k} x_j}{l+k}$, for $1 \leq k \leq r$. Here, au_k represents the *minimum* average utility of all pFEs $B = A \oplus E$ of A in t with $|E| = k \geq 1$. Then, $\min\{au_k, 1 \leq k \leq r\}$ represents the *minimum* average utility of all possible pFEs of A in t , and the tightest WLB on *auo*(A, t) should be this value, as presented as follows.

Definition 13 (The tightest WLB $twlbauo(A, t)$ for each supporting transaction t). Assuming that there exists an index p in X , $1 \leq p \leq r$, which satisfies the conditions $((\forall j = 1, \dots, p: x_j < au_{j-1}) \wedge (p = r \vee (p < r \wedge x_{p+1} \geq au_p)))$. If the such index does not hold, then by convention, let us set $p \stackrel{\text{def}}{=} 1$. Let $fmau(A, t) \stackrel{\text{def}}{=} \frac{u + \sum_{j=1..p} x_j}{l+p}$. Then, the tightest WLB $twlbauo(A, t)$ on *auo*(A, t) in each transaction $t \in \delta(A)$ is denoted and defined as $twlbauo(A, t) \stackrel{\text{def}}{=} \frac{fmau(A, t)}{u(t)}$.

Example 9. For $A = bc$, consider $t_5 \in \delta(A) = \{t_1, t_2, t_5\}$. Then, we obtain $rem(A, t_5) = \{(d, 8), (e, 165), (f, 20)\}$, so $r = 3$, $X = \{x_1, x_2, x_3\} = \{8, 20, 165\}$. We have $u = u(A, t_5) = 30$, $au_0 = au(A, t_5) = \frac{u}{2} = 15$, $au_1 = \frac{u+x_1}{l+1} = \frac{30+8}{2+1} = 12.67$, $au_2 = \frac{u+x_1+x_2}{l+2} = \frac{30+8+20}{2+2} = 14.5$, and $au_3 = \frac{u+x_1+x_2+x_3}{l+3} = \frac{30+8+20+165}{2+3} = 44.6$. Since $x_1 < au_0$ and $x_2 > au_1$, so $p = 1$ and $fmau(A, t_5) = au_1 = 12.67$. Thus, $twlbauo(A, t_5) = 12.67/227 = 0.056$.

Proposition 3 (The tightest WLB $twlbauo(A, t)$ on *auo*(A, t)). For any itemset A and transaction $t \in \delta(A)$, then
c. $twlbauo(A, t) = \min\{s_k, 1 \leq k \leq r\}$.
d. $twlbauo(A, t)$ is the *tightest* WLB on *auo*(A, t) in each transaction t .

Based on the tightest WLB $twlbauo(A, t)$ for each transaction t , the WLB $twlbauo(A)$ on *auo*(A) is designed as follows.

Definition 14 (Weak lower bound $twlbauo$ on *auo*). For any *strictly* frequent itemset $A \in \mathcal{FI}$, $|\delta(A)| \geq ms$, a new WLB on *auo* is denoted and defined as $twlbauo(A) \stackrel{\text{def}}{=} \frac{1}{ms} \sum_{i=1..ms} wg_i$.

where the series $\mathcal{F} \stackrel{\text{def}}{=} \{wg_i, i = 1..n\}$ that is obtained by sorting the series $\{twlbauo(A, t) \mid t \in \delta(A)\}$ in *ascending* order, and $n = |\delta(A)|$.

Example 10. Continuing with Example 6, for $A = bc$ and $ms = 2$, we have $twlbauo(A, t_1) = \left(\frac{20+112+8+42}{2+2}\right)/286 = 0.16$, $twlbauo(A, t_2) = \left(\frac{8+4+16}{2+1}\right)/268 = 0.035$ and $twlbauo(A, t_5) = 0.056$. Hence, $twlbauo(A) = (0.035 + 0.056)/2 = 0.046$.

The following theorem states that $twlbauo$ is the *tightest* WLB among all previously proposed (W)LBs on *auo*.

Theorem 4 (Properties of WLB $twlbauo$ on *auo*).

- a) $twlbauo$ is a WLB on *auo*, i.e. $twlbauo(A) \leq auo(B)$ for any strictly frequent itemset A and proper FE $B = A \oplus D$ of A with $D \neq \emptyset$.
- b) $twlbauo$ is the *tightest* WLB on *auo* (in each supporting row of the database).

Based on the proposed WLB, *twlbauo*, this paper introduces *the first novel pruning strategy* aimed at early elimination of *NCFHAUO* branches in the prefix search tree. This strategy leverages itemsets already generated in the *CFHAUOI* set and is presented as follows.

Theorem 5 (*Pruning Strategy 3 – PS3*). For the current itemset C being examined when constructing the *CFHAUOI* set, if there is a CFHAUOI $P \in \text{CFHAUOI}$ such that $P \supset C$, $\text{lastItem}(P) = \text{lastItem}(C)$, and $\text{supp}(C) = \text{supp}(P)$, then if $\text{twlbauo}(P) \geq mo$, the entire *NCFHAUO* proper branch $pBr(C)$ is pruned early. Additionally, if $\text{auo}(P) \geq mo$ or $C \notin \text{FHAUOI}$, the *NCFHAUO* node C is eliminated.

Corollary 1 (*Pruning Strategy 4 – PS4*). For any two nodes $C_1 = D \oplus x$ and $C_2 = D \oplus y$ having the same parent in the prefix tree such that $y \succ x$, when extending C_1 with item y to have the larger itemset $P = C_1 \oplus y$, if $\text{supp}(P) = \text{supp}(C_2)$ and $\text{twlbauo}(P) \geq mo$, then the whole *NCFHAUO* $pBr(C_2)$ proper branch is safely eliminated early. Additionally, the *NCFHAUO* C_2 node is also discarded, if $\text{auo}(P) \geq mo$ or $C_2 \notin \text{FHAUOI}$.

4.3. The proposed algorithm for mining closed FHAUOIs

Based on the proposed theoretical results, this subsection introduces a novel algorithm named **C-FHAUOIM** to efficiently discover the set *CFHAUOI*. The algorithm is developed to address the challenges of mining this set. To effectively implement the suggested algorithm, we utilize the **NVUV-list (or Utility-List)** data structure presented in [38].

The pseudo-code of the proposed algorithm is depicted in Fig. 1, with the objective of mining the *CFHAUOI*. It takes $\mathcal{D}'$, mo and ms as input. C-FHAUOIM initially scans $\mathcal{D}'$ to calculate the lauo_p and support values for items in $\mathcal{A}$ and determine the I set (line 1). Then, C-FHAUOIM removes irrelevant items from $\mathcal{A} \setminus I$ in $\mathcal{D}'$ to minimize the candidate set for extension (line 2). After sorting the I set in ascending order of item support, the algorithm scans the QDB $\mathcal{D}'$ again to construct the structure **NVUV-list** for items in I (lines 3-4). Then, with each item x in I , the Mine-CI procedure is invoked at line 7 to discover CFHAUOIs in the $pBr(x)$ branch. Finally, C-FHAUOIM offers the *CFHAUOI* set as its output (line 9).

C-FHAUOIM ($\mathcal{D}'$, mo , ms)

Input: the QDB $\mathcal{D}'$, minimum utility and support thresholds mo and ms .

Output: the set *CFHAUOI*.

1. **Scan** $\mathcal{D}'$ to calculate the support and WUB lauo_p of all items, and determine the set
 $I \triangleq \{x \in \mathcal{A} \mid \text{lauo}_p(x) \geq mo \wedge \text{supp}(x) \geq ms\};$ //Pruning strategy 1
2. **Remove** all irrelevant items in $\mathcal{A} \setminus I$ from $\mathcal{D}'$; // Pruning strategy 1
3. **Sort** I in ascending order of item support;
4. **Scan** $\mathcal{D}'$ again to construct the structure **NVUV-list** for all *relevant* items of I ;
5. $\text{CFHAUOI} = \emptyset$;
6. **for each** $x \in I$ **do** {
7. **Mine-CI**(x , mo , ms , CFHAUOI);
8. }
9. **return** CFHAUOI ;

Fig. 1. The C-FHAUOIM algorithm.

The Mine-CI procedure depicted in Fig. 2 first verifies if $\text{supp}(C)$ is less than ms . If yes, it stops immediately (line 1). Then, it checks if the node C or the $pBr(C)$ proper branch is pruned by Pruning Strategy 3 in Theorem 5 or not (line 2-6). If yes, the fields *Prune_NC_Node* and/or *Prune_NC_pBr* of itemset C are marked to prune in subsequent steps. Next, if C is an FHAUOI ($\text{auo}(C) \geq mo$) and the value of its field *Prune_NC_Node* is false (lines 7-8), then the procedure Update-Closed-FHAUOI in Fig. 4 is called at lines 9-10 to update the *CFHAUOI* set. After that, Mine-CI evaluates whether any of the conditions outlined at line 12 are met. If affirmative, the $pBr(C)$ is pruned prematurely (line 13). In the case of negative, a new set of items, newI , is determined for expanding C (line 17), utilizing the criteria specified at line 16. Subsequently, each item from newI is added to extend the itemset C , generating a new itemset $P = Cy$, and establishing the NVUV-list structure of P at line 20. The NCFHAUO-Branch-Pruning procedure illustrated in Fig. 3 is invoked to identify *NCFHAUO*

branches and nodes for subsequent pruning steps (line 21). Ultimately, Mine-CI calls itself recursively to explore larger CFHAUOIs (line 24).

```

Mine-CI ( $C, mo, ms, CFHAUOI$ )
  Input: the itemset  $C$ ,  $mo$  and  $ms$  thresholds, and set  $CFHAUOI$ .
  Output:  $CFHAUOI$  is updated.
1. if ( $supp(C) < ms$ ) return;
2. for each  $P \in CFHAUOI$  and  $P \supset C$  do {
3.   NCFHAUO-Branch-Pruning ( $C, P$ );    // Pruning strategy 3
4.   If ( $C.Prune\_NC\_pBr = true$ ) then
5.     break;
6. }
7. if ( $auo(C) \geq mo$ ) then {
8.   if ( $C.Prune\_NC\_Node = false$ ) then
9.     Update-Closed-FHAUOI ( $C, CFHAUOI$ );
10. }
11. if ( $\tau_{\delta}(C) < mo$  or  $C.Prune\_NC\_pBr = true$ )    //Pruning strategies 2-4
12.   return;
13.  $x = lastItem(C)$ ;  $D = prefix(C)$ ;  $newI = \emptyset$ ;
14. for each  $y \in I$  such that  $y > x$  do {
15.   if ( $lauo_{\rho}(Cy) \geq mo$  and  $supp(Cy) \geq ms$ ) then    //Pruning strategy 1
16.      $newI = newI \cup \{y\}$ ;
17.  $Exts\_of\_C = \emptyset$ ;
18. for each  $y \in newI$  do {
19.   Add  $P = Cy$  to  $Exts\_of\_C$  and construct the structure NVUV-list of  $P$ ;
20.   NCFHAUO-Branch-Pruning ( $Dy, P$ );    // Pruning strategy 4
21. }
22. for each  $P \in Exts\_of\_C$  do
23.   Mine-CI ( $P, mo, ms, CFHAUOI$ );

```

Fig. 2. The Mine-CI procedure.

```

NCFHAUO-Branch-Pruning ( $C, P$ )
  Input: the itemsets  $C$  and  $P$ .
  Output:  $C$  and  $P$  have been updated.
1. if ( $supp(C) = supp(P)$ ) then {
2.   if ( $lastItem(C) = lastItem(P)$ ) then {
3.     if ( $twlbauo(P) \geq mo$ ) then {
4.        $C.Prune\_NC\_pBr = true$ ;    // Pruning strategy 4
5.       if ( $auo(P) \geq mo$  or  $auo(C) < mo$ ) then
6.          $C.Prune\_NC\_Node = true$ ;    // Pruning strategy 4
7.     }
8.   }
9. }

```

Fig. 3. The NCFHAUO-Branch-Pruning procedure.

```

Update-Closed-FHAUOI ( $C, CFHAUOI$ )
  Input: the FHAUOI itemset  $C$  and the set  $CFHAUOI$ .
  Output:  $CFHAUOI$  has been updated.
1. for each  $P \in CFHAUOI$  do
2.   if ( $C \subset P$  and  $supp(C) = supp(P)$ ) then
3.     return;
4. for each  $R \in CFHAUOI$  do
5.   if ( $R \subset C$  and  $supp(C) = supp(R)$ ) then
6.     Remove  $R$  from  $CFHAUOI$ ;
7.    $CFHAUOI := CFHAUOI \cup \{C\}$ ;

```

Fig. 4. The Update-Closed-FHAUOI procedure.

5. Experimental Evaluation

6. Conclusions and future work

References